**UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR**

Faculté des Sciences et Techniques

Département de Mathématiques et Informatique

Section Informatique

Master II Informatique

Option **Business Intelligence**

Rapport de projet

# Conception d'une Modern Data Platform avec architecture Médaillon

## et développement d'une application Text-to-SQL basée sur l'IA (Snowflake Cortex)

*SQL Server · DLT · Snowflake · dbt · Dagster · Streamlit · Cortex Analyst*

**Réalisé par :**

Mouhamed DIOP

**Sous la direction de :**

Dr Jean Gane SARR

Année universitaire 2025 – 2026

# Remerciements

Je tiens à exprimer ma profonde gratitude à **Dr Jean Gane SARR**, enseignant au Département de Mathématiques et Informatique de la Faculté des Sciences et Techniques, pour la proposition de ce sujet, la qualité de son encadrement et la pertinence de ses orientations tout au long de ce projet.

Mes remerciements s'adressent également à l'ensemble du corps professoral de la section Informatique de l'Université Cheikh Anta Diop de Dakar, et particulièrement aux enseignants du Master II option Business Intelligence, pour les connaissances transmises durant ce cursus.

Je remercie enfin ma famille et mes camarades de promotion pour leur soutien et leurs encouragements.

# Résumé

Les organisations disposent de volumes croissants de données opérationnelles, mais leur exploitation reste freinée par la complexité technique des outils d'interrogation : sans maîtrise du langage SQL, les décideurs dépendent des équipes techniques pour obtenir le moindre indicateur.

Ce projet met en œuvre une **Modern Data Platform** complète sur le jeu de données de référence *Wide World Importers* de Microsoft. Les données transactionnelles de **SQL Server** sont ingérées de manière incrémentale dans **Snowflake** par **DLT**, puis organisées selon une **architecture Médaillon** : une couche *Bronze* brute (30 tables), une couche *Silver* nettoyée (29 modèles) et une couche *Gold* modélisée en étoile (15 tables, dont trois historisées en SCD de type 2). Les transformations sont développées, documentées et testées avec **dbt** (189 nœuds, dont plus de 140 tests de qualité), et l'ensemble est orchestré par **Dagster** (75 assets, 146 contrôles, exécution quotidienne avec reprise sur erreur et alertes).

Une application **Streamlit** permet enfin aux décideurs d'interroger la couche Gold **en langage naturel** : les questions, posées en français, sont traduites en SQL par **Snowflake Cortex Analyst** grâce à un modèle sémantique décrivant les tables, leurs relations, leurs synonymes métier et leurs indicateurs. La requête générée est contrôlée (lecture seule), exécutée avec un rôle aux droits restreints, puis restituée sous forme de tableau et de graphique. L'application est déployée à la fois en local et dans *Streamlit in Snowflake*, et une chaîne d'intégration continue GitHub Actions valide chaque modification sur un clone « zéro copie » de l'entrepôt.

**Mots-clés :** Modern Data Stack, architecture Médaillon, Snowflake, DLT, dbt, Dagster, modélisation dimensionnelle, SCD2, Text-to-SQL, Cortex Analyst, LLM, Streamlit.

# Abstract

Organisations accumulate ever-growing volumes of operational data, yet business users who do not master SQL still depend on technical teams to obtain even simple indicators.

This project builds an end-to-end **Modern Data Platform** on Microsoft's *Wide World Importers* sample database. Transactional data from **SQL Server** is incrementally ingested into **Snowflake** with **DLT** and organised following a **Medallion architecture** (Bronze, Silver and a Gold star schema including type-2 slowly changing dimensions). Transformations are versioned, documented and tested with **dbt**, and the whole pipeline is orchestrated by **Dagster** with daily scheduling, retries, freshness checks and failure alerts.

A **Streamlit** application lets decision makers query the Gold layer in **natural language**: French questions are translated into SQL by **Snowflake Cortex Analyst** using a semantic model, executed under a read-only role, and rendered as tables and charts. The application runs locally and in *Streamlit in Snowflake*, and a GitHub Actions pipeline validates every change on a zero-copy clone of the warehouse.

**Keywords:** Modern Data Stack, Medallion architecture, Snowflake, DLT, dbt, Dagster, dimensional modelling, SCD2, Text-to-SQL, Cortex Analyst, LLM, Streamlit.

# Liste des sigles et abréviations

| Sigle | Signification |
|---|---|
| BI | Business Intelligence (informatique décisionnelle) |
| CA | Chiffre d'affaires |
| CI/CD | Intégration continue / Déploiement continu |
| DAG | Directed Acyclic Graph (graphe orienté acyclique) |
| dbt | data build tool |
| DLT | data load tool |
| ELT | Extract, Load, Transform |
| ETL | Extract, Transform, Load |
| HT / TTC | Hors taxes / Toutes taxes comprises |
| KPI | Key Performance Indicator (indicateur clé de performance) |
| LLM | Large Language Model (grand modèle de langage) |
| MDP | Modern Data Platform |
| OLAP | Online Analytical Processing |
| OLTP | Online Transaction Processing |
| RBAC | Role-Based Access Control |
| SCD | Slowly Changing Dimension (dimension à évolution lente) |
| SiS | Streamlit in Snowflake |
| SQL | Structured Query Language |
| TVA | Taxe sur la valeur ajoutée |
| WWI | Wide World Importers |
| YAML | YAML Ain't Markup Language |

# Table des matières

**Remerciements** i

**Résumé** ii

**Abstract** iii

**Liste des sigles et abréviations** iv

**Introduction générale** 1

**1 Contexte, problématique et objectifs** 2

1.1 Contexte du projet . . . 2

1.2 Problématique . . . 2

1.3 Objectifs . . . 2

1.3.1 Objectif général . . . 2

1.3.2 Objectifs spécifiques . . . 3

1.4 Le jeu de données Wide World Importers . . . 3

**2 Concepts et technologies** 5

2.1 De l'ETL à la Modern Data Stack . . . 5

2.2 L'architecture Médaillon . . . 5

2.3 La modélisation dimensionnelle . . . 6

2.3.1 Dimensions à évolution lente (SCD) . . . 6

2.4 Les outils de la plateforme . . . 6

2.4.1 SQL Server . . . 6

2.4.2 Snowflake . . . 6

2.4.3 DLT (data load tool) . . . 7

2.4.4 dbt (data build tool) . . . 7

2.4.5 Dagster . . . 7

2.4.6 Streamlit . . . 7

2.5 Le Text-to-SQL et Snowflake Cortex Analyst . . . 7

**3 Architecture et conception** 9

3.1 Architecture globale . . . 9

3.2 Organisation de l'environnement Snowflake . . . 10

3.2.1 Objets . . . 10

3.2.2 Sécurité par les rôles . . . 10
3.3 Conception du modèle de données . . . 11
3.3.1 Couche Bronze . . . 11
3.3.2 Couche Silver . . . 11
3.3.3 Couche Gold : schéma en étoile . . . 11

**4 Réalisation de la plateforme de données 12**
4.1 Mise en place du système source . . . 12
4.2 Ingestion dans la couche Bronze avec DLT . . . 12
4.2.1 Stratégie de chargement . . . 12
4.2.2 Traitements particuliers . . . 13
4.2.3 Résultat et réconciliation . . . 13
4.3 Couche Silver : nettoyage et normalisation . . . 13
4.4 Couche Gold : le schéma en étoile . . . 14
4.5 Historisation des données (SCD de type 2) . . . 16
4.5.1 Exploitation des tables temporelles de SQL Server . . . 16
4.5.2 Snapshot dbt du niveau de stock . . . 16
4.6 Qualité des données . . . 16
4.7 Orchestration avec Dagster . . . 17
4.7.1 Assets et lignage . . . 17
4.7.2 Planification et reprise sur erreur . . . 18

**5 Application Text-to-SQL avec Cortex AI 19**
5.1 Choix du service : Cortex Analyst . . . 19
5.2 Le modèle sémantique . . . 19
5.3 Fonctionnement de l'application . . . 20
5.3.1 Architecture du code . . . 20
5.3.2 Conversation multi-tours . . . 21
5.3.3 Choix automatique de la visualisation . . . 21
5.4 Sécurité de la génération de SQL . . . 21
5.5 Interface utilisateur . . . 21

**6 Industrialisation de la plateforme 24**
6.1 Traitements incrémentaux . . . 24
6.2 Alertes et supervision . . . 25
6.2.1 Alertes Dagster . . . 25
6.2.2 Maîtrise des coûts Snowflake . . . 25
6.3 Déploiement dans Streamlit in Snowflake . . . 25
6.4 Intégration et déploiement continus . . . 25

**7 Résultats et évaluation** **27**

7.1 Bilan quantitatif de la plateforme . . . . . 27

7.2 Évaluation de l'application Text-to-SQL . . . . . 27

7.3 Quelques enseignements métier . . . . . 28

7.4 Difficultés rencontrées et solutions . . . . . 28

7.5 Limites . . . . . 29

**Conclusion générale et perspectives** **30**

**A Structure du dépôt** **31**

**B Guide d'installation et commandes** **32**

**C Extraits de code complémentaires** **33**

**Bibliographie et webographie** **34**

# Table des figures

2.1 Principe de l'architecture Médaillon . . . . . 5

3.1 Architecture globale de la Modern Data Platform . . . . . 9
3.2 Hiérarchie des rôles Snowflake (une flèche signifie « est accordé à ») . . . . . 10
3.3 Schéma en étoile de la couche Gold (jaune : faits, bleu : dimensions, violet : historisation) . . . . . 11

4.1 Lignage amont de `gold/fact_sales` dans Dagster (Bronze → Silver → Gold) . . 18
4.2 Automatisations actives dans Dagster : planning quotidien et capteurs d'alerte . 18

5.1 Traitement d'une question en langage naturel . . . . . 20
5.2 Page d'accueil de l'application : en-tête UCAD, exemples de questions et pied de page . . . . . 22
5.3 « Quel est le montant total des ventes par mois ? » : courbe générée automatiquement 22
5.4 « Quel est le chiffre d'affaires par catégorie de client et par année ? » : une série par catégorie . . . . . 23

6.1 Chaîne d'intégration et de déploiement continus . . . . . 26

# Liste des tableaux

1.1 Schémas métier de la base WideWorldImporters (OLTP) . . . . . 3

3.1 Rôle de chaque composant et versions utilisées . . . . . 10

4.1 Stratégie de chargement par type de table . . . . . 12
4.2 Volumes des principales tables Bronze (réconciliées avec SQL Server) . . . . . 13
4.3 Règles appliquées dans la couche Silver . . . . . 14
4.4 Tables de la couche Gold . . . . . 15
4.5 Typologie des tests de qualité . . . . . 17

6.1 Modèles incrémentaux . . . . . 24

7.1 Indicateurs de la plateforme réalisée . . . . . 27
7.2 Questions testées et résultats obtenus . . . . . 27
7.3 Principales difficultés et solutions apportées . . . . . 28

# Liste des listings


4.1 Choix automatique de la stratégie de chargement (extrait) . . . . . 12
4.2 Macros de nettoyage (`macros/cleaning.sql`, extrait) . . . . . 13
4.3 Table de faits des ventes (`fact_sales.sql`, extrait) . . . . . 15
4.4 Snapshot dbt du niveau de stock (`snapshots.yml`) . . . . . 16
4.5 Test de réconciliation Bronze / Gold . . . . . 17
5.1 Extrait du modèle sémantique : table des ventes et métrique de marge . . . . . 19
5.2 Contrôle de lecture seule du SQL généré . . . . . 21
A.1 Arborescence du projet . . . . . 31
C.1 Assets Bronze : relier DLT et dbt dans le lignage Dagster . . . . . 33
C.2 Job quotidien avec reprise sur erreur . . . . . 33
C.3 Dimension produit historisée (SCD2) . . . . . 33
C.4 Appel de Cortex Analyst dans Streamlit in Snowflake . . . . . 33

# Introduction générale

Les entreprises produisent aujourd'hui de très grandes quantités de données issues de leurs systèmes opérationnels : ventes, achats, stocks, paiements, logistique. Ces données constituent une ressource stratégique pour suivre l'activité, mesurer la performance et éclairer la prise de décision. Leur exploitation demeure pourtant difficile : elles sont dispersées dans des bases transactionnelles conçues pour l'enregistrement des opérations et non pour l'analyse, et leur interrogation exige une maîtrise technique, en particulier du langage SQL, que la plupart des décideurs ne possèdent pas.

Deux évolutions récentes changent profondément cette situation. D'une part, l'émergence de la *Modern Data Stack* — entrepôts de données dans le cloud, outils d'ingestion déclaratifs, transformations versionnées et testées, orchestrateurs orientés données — permet d'industrialiser la chaîne de valeur de la donnée avec les pratiques du génie logiciel. D'autre part, les progrès des grands modèles de langage (LLM) rendent possible l'interrogation des données en langage naturel : une question posée en français peut être traduite automatiquement en une requête SQL exécutable.

C'est dans ce double contexte que s'inscrit ce projet, réalisé dans le cadre du Master II Informatique, option Business Intelligence, de l'Université Cheikh Anta Diop de Dakar. Il vise à concevoir et à mettre en œuvre une **Modern Data Platform** reposant sur une **architecture Médaillon** (Bronze, Silver, Gold) dans Snowflake, alimentée depuis SQL Server par **DLT**, transformée avec **dbt** et orchestrée par **Dagster**, puis à développer une application **Streamlit** de type *Text-to-SQL* s'appuyant sur **Snowflake Cortex AI** pour permettre aux décideurs d'obtenir leurs indicateurs sans écrire une ligne de SQL.

Le présent rapport est organisé comme suit :

— le **chapitre 1** présente le contexte, la problématique, les objectifs et le jeu de données Wide World Importers ;

— le **chapitre 2** expose les concepts et technologies mobilisés ;

— le **chapitre 3** décrit l'architecture retenue et la conception du modèle de données ;

— le **chapitre 4** détaille la réalisation de la plateforme : ingestion, couches Silver et Gold, historisation, qualité des données et orchestration ;

— le **chapitre 5** est consacré à l'application Text-to-SQL ;

— le **chapitre 6** traite de l'industrialisation : traitements incrémentaux, alertes, déploiement dans Snowflake et intégration continue ;

— le **chapitre 7** présente les résultats obtenus, les difficultés rencontrées et leurs solutions ;

— enfin, la conclusion dresse le bilan du projet et en propose les perspectives.

CHAPITRE 1

# Contexte, problématique et objectifs

## 1.1 Contexte du projet

Dans la plupart des organisations, les données de gestion sont stockées dans des bases relationnelles transactionnelles (OLTP). Ces bases sont optimisées pour enregistrer rapidement de nombreuses petites opérations — une commande, une facture, un paiement — et sont donc fortement normalisées : une simple question comme « quel est le chiffre d'affaires par catégorie de client et par année ? » nécessite des jointures entre une demi-douzaine de tables, des règles de calcul (hors taxes, toutes taxes comprises, avoirs) et une bonne connaissance du schéma.

L'informatique décisionnelle répond traditionnellement à ce problème par la construction d'un entrepôt de données (*data warehouse*) alimenté par des traitements ETL et exploité au moyen de rapports et de tableaux de bord. Cette approche reste cependant rigide : chaque nouveau besoin d'analyse passe par l'équipe technique, qui doit écrire la requête ou faire évoluer le rapport.

## 1.2 Problématique

Le sujet formule la problématique suivante : *comment concevoir une plateforme de données moderne capable d'automatiser l'ingestion, la transformation et la modélisation des données, tout en offrant une interface conversationnelle permettant d'interroger ces données en langage naturel grâce à l'intelligence artificielle ?*

Cette problématique soulève plusieurs questions techniques :

— comment extraire de manière fiable et **incrémentale** les données d'un système source transactionnel vers un entrepôt dans le cloud ?

— comment organiser les données en couches successives garantissant la **traçabilité**, la **qualité** et la **facilité d'analyse** ?

— comment automatiser, surveiller et fiabiliser l'enchaînement des traitements ?

— comment permettre à un LLM de générer un SQL **correct** sur un schéma métier, et comment garantir que cette génération reste **sûre** (aucune modification possible des données) ?

## 1.3 Objectifs

### 1.3.1 Objectif général

Concevoir et mettre en œuvre une Modern Data Platform centralisant les données de l'entreprise, les transformant selon une architecture Médaillon, et exposant une application Text-to-SQL capable de convertir automatiquement les questions des utilisateurs en requêtes SQL.

### 1.3.2 Objectifs spécifiques

1. Concevoir une architecture moderne de données de bout en bout.
2. Automatiser les pipelines d'ingestion de SQL Server vers Snowflake avec DLT.
3. Mettre en place les couches Bronze, Silver et Gold dans Snowflake.
4. Transformer, documenter et tester les données avec dbt.
5. Orchestrer l'ensemble des traitements avec Dagster.
6. Développer une interface utilisateur avec Streamlit.
7. Intégrer Snowflake Cortex AI pour convertir les questions en langage naturel en requêtes SQL exécutables.

Au-delà de ces objectifs, le projet traite les pistes d'amélioration identifiées en cours de réalisation : historisation des données, modèles incrémentaux, déploiement dans Streamlit in Snowflake, alertes et intégration continue.

## 1.4 Le jeu de données Wide World Importers

*Wide World Importers* (WWI) est la base d'exemple de référence publiée par Microsoft pour SQL Server depuis la version 2016. Elle modélise l'activité d'une entreprise fictive de distribution en gros d'articles de nouveauté (jouets, mugs, t-shirts, articles d'emballage, gadgets USB…) basée aux États-Unis, entre janvier 2013 et mai 2016.

Microsoft fournit deux sauvegardes :

— `WideWorldImporters-Full.bak` : la base **transactionnelle (OLTP)** ;

— `WideWorldImportersDW-Full.bak` : un entrepôt de données déjà modélisé (OLAP).

Le projet utilise volontairement la base **OLTP** : elle joue le rôle du système source, et c'est la plateforme développée qui reconstruit l'entrepôt analytique. Utiliser la version DW aurait vidé le travail de modélisation de sa substance.

La base comprend quatre schémas métier, présentés dans le tableau 1.1.

**Table 1.1** – Schémas métier de la base WideWorldImporters (OLTP)

| Schéma | Contenu et principales tables | Lignes (max.) |
|---|---|---|
| `Sales` | Clients, commandes, factures, transactions clients (`Invoices`, `InvoiceLines`, `Orders`, `OrderLines`, `Customers`) | 231 412 |
| `Purchasing` | Fournisseurs, bons de commande, transactions fournisseurs | 8 367 |
| `Warehouse` | Articles, stocks, mouvements de stock, groupes d'articles | 236 667 |
| `Application` | Référentiels : villes, provinces, pays, personnes, modes de livraison et de paiement | 37 940 |

Plusieurs caractéristiques de WWI en font un cas d'étude réaliste et exigeant :

— des **tables temporelles** (*system-versioned*) qui conservent l'historique de chaque modification dans des tables d'archive (`*_Archive`) ;

— une colonne `LastEditedWhen` sur les tables transactionnelles, exploitable pour l'ingestion incrémentale ;

— des types de données spécifiques à SQL Server (`geography`, `varbinary`) et des champs JSON (`CustomFields`) ;

— des tables *In-Memory OLTP* (optimisées en mémoire).

CHAPITRE 2

# Concepts et technologies

## 2.1 De l'ETL à la Modern Data Stack

Les chaînes décisionnelles classiques reposent sur le modèle **ETL** (*Extract, Transform, Load*) : les données sont transformées par un outil dédié avant d'être chargées dans l'entrepôt. Avec l'arrivée des entrepôts de données dans le cloud, dont la puissance de calcul est élastique, le modèle **ELT** s'est imposé : les données sont d'abord chargées brutes dans l'entrepôt, puis transformées *dans* l'entrepôt, en SQL.

La **Modern Data Stack** désigne l'ensemble d'outils spécialisés et interopérables qui accompagnent ce modèle :

— un **entrepôt cloud** séparant stockage et calcul (Snowflake, BigQuery, Redshift…) ;

— un outil d'**ingestion** déclaratif (Fivetran, Airbyte, DLT…) ;

— un outil de **transformation** versionné et testé (dbt) ;

— un **orchestrateur** (Airflow, Dagster, Prefect) ;

— des outils de **restitution** (tableaux de bord, applications analytiques, IA).

Cette approche applique à la donnée les pratiques du génie logiciel : gestion de versions, tests automatisés, revue de code, intégration continue et déploiement reproductible.

## 2.2 L'architecture Médaillon

L'architecture Médaillon, popularisée par Databricks, organise les données d'un *lakehouse* ou d'un entrepôt en trois couches de qualité croissante (figure 2.1).




**Figure 2.1** – Principe de l'architecture Médaillon

— **Bronze** : les données sont chargées telles quelles depuis la source. Cette couche garantit la traçabilité et permet de rejouer les transformations sans réinterroger la source.

— **Silver** : les données sont nettoyées, typées, dédoublonnées et conformées. Elles restent proches de la structure source mais sont fiables.

— **Gold** : les données sont modélisées pour un usage métier — ici un schéma en étoile — et directement exploitables par les outils d'analyse et par l'IA.

## 2.3 La modélisation dimensionnelle

La couche Gold suit la **modélisation dimensionnelle** de Ralph Kimball, qui organise les données autour :

— de **tables de faits**, qui enregistrent des événements mesurables (une ligne de facture, un mouvement de stock) avec leurs mesures numériques (quantité, montant, bénéfice) ;

— de **tables de dimensions**, qui décrivent le contexte de ces événements (qui, quoi, où, quand) avec des attributs descriptifs (nom du client, catégorie, ville, mois…).

Le **grain** d'une table de faits — le niveau de détail d'une ligne — est la décision de conception centrale. Ce modèle simplifie fortement les requêtes analytiques : un indicateur s'obtient par agrégation des faits et regroupement selon des attributs de dimensions, avec des jointures simples et prévisibles. Cette propriété est précieuse pour la génération automatique de SQL par un LLM.

### 2.3.1 Dimensions à évolution lente (SCD)

Les attributs des dimensions changent dans le temps : un produit change de prix, un client de plafond de crédit. Kimball propose plusieurs stratégies, dont :

— **SCD de type 1** : l'ancienne valeur est écrasée (pas d'historique) ;

— **SCD de type 2** : chaque changement crée une nouvelle ligne, avec une période de validité (`valid_from`, `valid_to`) et un indicateur de version courante.

## 2.4 Les outils de la plateforme

### 2.4.1 SQL Server

SGBD relationnel de Microsoft, utilisé ici comme **système source** transactionnel hébergeant la base WideWorldImporters (SQL Server 2025, édition Developer).

### 2.4.2 Snowflake

Snowflake est un entrepôt de données proposé en SaaS sur les principaux clouds. Il sépare le **stockage** (facturé au volume) du **calcul**, assuré par des *virtual warehouses* dimensionnables, suspendus automatiquement lorsqu'ils sont inactifs. Il offre notamment :

— un contrôle d'accès par rôles (RBAC) hiérarchiques ;

— le **clonage zéro copie** (*zero-copy cloning*) d'une base entière, instantané et sans duplication physique des données ;

— la *Time Travel* et des *resource monitors* pour maîtriser la consommation ;

— **Snowflake Cortex**, une suite de services d'IA exécutés au sein de la plateforme, dont **Cortex Analyst** pour le Text-to-SQL ;

— **Streamlit in Snowflake**, l'hébergement d'applications Streamlit directement dans le compte, sans infrastructure à gérer.

### 2.4.3 DLT (data load tool)

DLT est une bibliothèque Python open source d'ingestion de données. Elle extrait les données d'une source (ici une base SQL via SQLAlchemy), infère et fait évoluer le schéma cible, normalise les noms, et charge les données dans la destination. Elle gère nativement le **chargement incrémental** (curseur sur une colonne de date) et les modes d'écriture `append`, `replace` et `merge` (fusion sur clé primaire), ainsi que la conservation de l'état entre deux exécutions.

### 2.4.4 dbt (data build tool)

dbt permet d'écrire les transformations sous forme de modèles `SELECT` en SQL enrichi de Jinja. dbt résout les dépendances entre modèles (fonction `ref()`), les matérialise dans l'entrepôt (vues, tables, tables incrémentales), exécute des **tests** de qualité (unicité, non-nullité, intégrité référentielle, tests personnalisés), génère la **documentation** et le **lignage**, et propose des **snapshots** pour historiser des données en SCD de type 2.

### 2.4.5 Dagster

Dagster est un orchestrateur orienté *assets* : au lieu de décrire des tâches, on déclare les **actifs de données** produits (tables, modèles) et leurs dépendances. Dagster en déduit le graphe d'exécution, offre une interface de visualisation du lignage, des planifications (*schedules*), des capteurs (*sensors*), des politiques de reprise sur erreur, et des contrôles de qualité (*asset checks*). Ses intégrations `dagster-dlt` et `dagster-dbt` transforment automatiquement chaque table DLT et chaque modèle dbt en asset, reliés dans un même lignage.

Le sujet laissait le choix entre Dagster et Airflow ; Dagster a été retenu car il est explicitement recommandé par l'énoncé, que son approche par assets correspond naturellement à une architecture en couches, et qu'il s'intègre nativement avec DLT et dbt.

### 2.4.6 Streamlit

Streamlit est un cadriciel Python permettant de construire rapidement des applications web de données interactives, sans développement front-end : composants de saisie, tableaux, graphiques, interface de conversation (*chat*).

## 2.5 Le Text-to-SQL et Snowflake Cortex Analyst

Le **Text-to-SQL** consiste à traduire une question en langage naturel en une requête SQL exécutable. Les grands modèles de langage ont fait progresser considérablement cette tâche, mais une génération « naïve » — envoyer la question et la liste des colonnes à un LLM — reste fragile : ambiguïté des termes métier (« ventes » : HT ou TTC ?), jointures erronées, colonnes inventées, calculs incorrects.

**Cortex Analyst** est le service de Snowflake dédié à ce problème. Il s'appuie sur un **modèle sémantique** (fichier YAML) qui décrit :

— les tables logiques et leurs colonnes, classées en *dimensions*, *dimensions temporelles*, *faits* et *métriques* ;

— des descriptions et des **synonymes** métier (par exemple « CA » pour le chiffre d'affaires) ;

— les **relations** (jointures) entre tables ;

— des **requêtes vérifiées** (*verified queries*) servant d'exemples de référence ;

— des instructions personnalisées (langue de réponse, conventions de calcul).

Cortex Analyst s'expose sous forme d'API REST : il reçoit la conversation et le modèle sémantique, et renvoie une interprétation de la question, la requête SQL générée ou, en cas d'ambiguïté, des suggestions de reformulation. Il ne fait qu'*écrire* le SQL : son exécution reste sous le contrôle de l'application, avec les droits de l'utilisateur.

CHAPITRE 3

# Architecture et conception

## 3.1 Architecture globale

La figure 3.1 présente l'architecture retenue. Elle suit le cheminement de la donnée, du système source jusqu'au décideur.

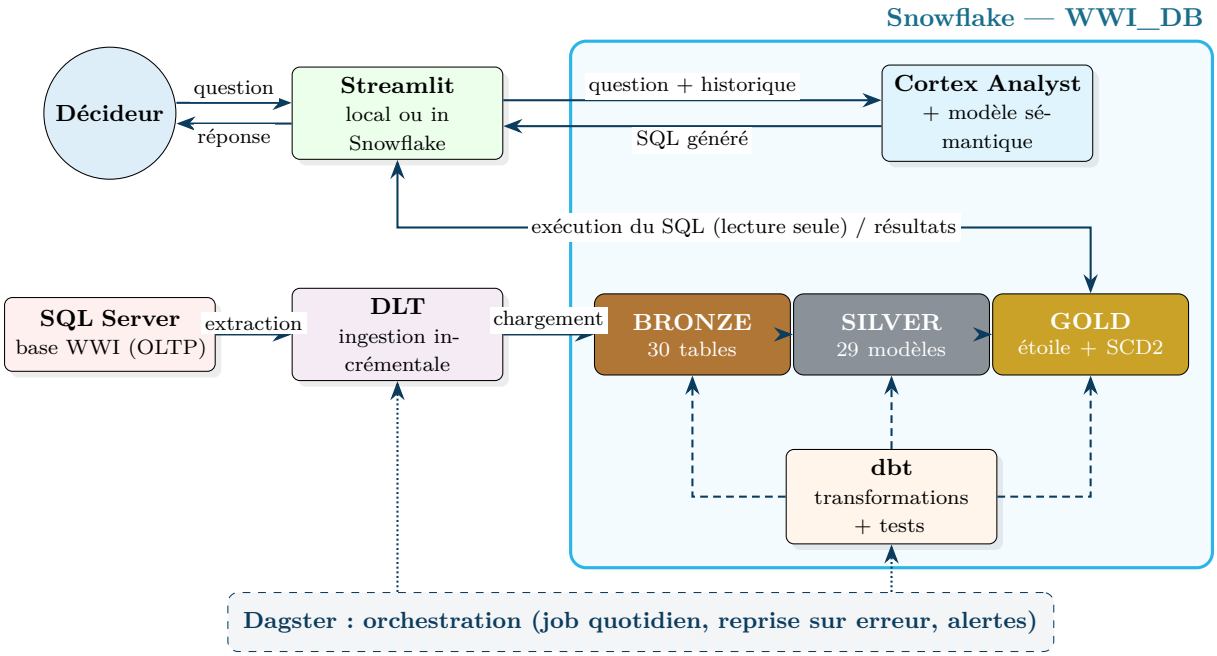



**Figure 3.1** – Architecture globale de la Modern Data Platform

**Table 3.1** – Rôle de chaque composant et versions utilisées

| Étape | Outil (version) | Rôle |
|---|---|---|
| Source | SQL Server 2025 Developer | Base OLTP WideWorldImporters |
| Ingestion | DLT 1.31 | Extraction SQL Server → Snowflake BRONZE, incrémental + merge |
| Stockage | Snowflake (AWS US West 2) | Base `WWI_DB`, schémas BRONZE, SILVER, GOLD, APP |
| Transformation | dbt-core 1.12 | Modèles SQL versionnés, tests, snapshots, documentation |
| Orchestration | Dagster 1.13 | Lignage DLT → dbt, planning, reprise, alertes |
| IA | Snowflake Cortex Analyst | Traduction question → SQL sur la couche Gold |
| Interface | Streamlit 1.66 | Conversation, SQL généré, graphiques, export CSV |
| CI/CD | GitHub Actions | Lint, `dbt build` sur clone, déploiement de l'application |

## 3.2 Organisation de l'environnement Snowflake

### 3.2.1 Objets

Un script d'initialisation (`snowflake/00_setup.sql`) crée l'ensemble des objets :

— l'entrepôt de calcul `WWI_WH`, de taille XS, suspendu automatiquement après 60 secondes d'inactivité afin d'économiser les crédits ;

— la base `WWI_DB` et ses schémas `BRONZE`, `SILVER`, `GOLD`, ainsi que `APP` pour l'application ;

— l'utilisateur technique `WWI_PIPELINE_USER` utilisé par les pipelines.

### 3.2.2 Sécurité par les rôles

Le principe du **moindre privilège** est appliqué au moyen de deux rôles (figure 3.2).

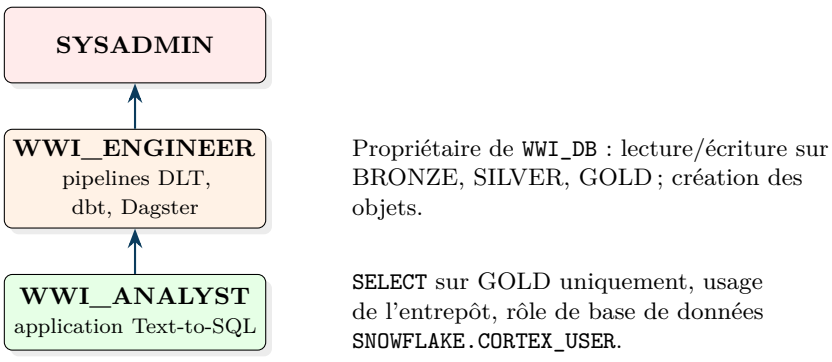



**Figure 3.2** – Hiérarchie des rôles Snowflake (une flèche signifie « est accordé à »)

L'application Text-to-SQL s'exécute exclusivement avec `WWI_ANALYST` : même si le modèle de langage générait une requête malveillante, le rôle n'aurait pas le droit de modifier ni même de

lire les couches Bronze et Silver.

## 3.3 Conception du modèle de données

### 3.3.1 Couche Bronze

La couche Bronze reproduit fidèlement les tables sources. Les tables sont nommées `<schéma>_<table>` en *snake_case* (`Sales.InvoiceLines` devient `sales_invoice_lines`) afin d'éviter les collisions entre schémas et de conserver l'origine de chaque table.

### 3.3.2 Couche Silver

Chaque table Silver (préfixe `slv_`) correspond à une table Bronze, nettoyée selon des règles communes factorisées dans des macros dbt (section 4.3). Deux modèles supplémentaires reconstituent l'historique complet des produits et des clients.

### 3.3.3 Couche Gold : schéma en étoile

La couche Gold est un schéma en étoile (constellation de faits) centré sur six processus métier (figure 3.3).

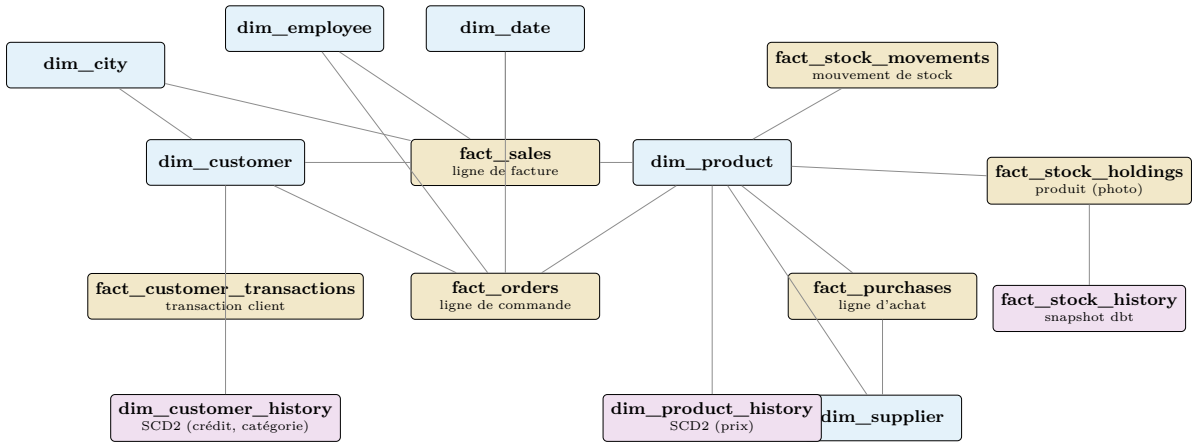



**Figure 3.3** – Schéma en étoile de la couche Gold (jaune : faits, bleu : dimensions, violet : historisation)

Les principaux choix de conception sont les suivants :

— **grain fin** : les faits de ventes, commandes et achats sont au niveau de la ligne de document, ce qui permet tous les agrégats ;

— **clés naturelles** : les identifiants de la source (`customer_id`, `stock_item_id`...) servent de clés. Ce choix simplifie les jointures générées par l'IA ; l'historisation est portée par des tables SCD2 dédiées, dotées de clés de substitution ;

— **dimensions dénormalisées** : la géographie (ville → province → pays → continent) est aplatie dans `dim_city` et reprise dans `dim_customer` ;

— **dimensions dégénérées** : les attributs de faible cardinalité (mode de livraison, type de transaction, moyen de paiement) sont stockés directement dans les faits ;

— **libellés français** dans `dim_date` (mois et jours) pour faciliter la lecture des résultats.

CHAPITRE 4

# Réalisation de la plateforme de données

## 4.1 Mise en place du système source

La sauvegarde `WideWorldImporters-Full.bak` (121 Mo) a été restaurée sur une instance locale de SQL Server 2025 Developer. Deux obstacles ont été rencontrés :

— le service SQL Server, exécuté sous un compte de service, n'avait pas le droit de lire le dossier *Téléchargements* de l'utilisateur (« *Operating system error 5 : Accès refusé* ») : la sauvegarde a été copiée dans un dossier partagé (`C:\Users\Public\SQLBackups`) ;

— le pilote ODBC 18 chiffre les connexions par défaut et refusait le certificat auto-signé de l'instance locale : l'option `-C` (`TrustServerCertificate`) a été ajoutée.

La restauration a converti la base au format interne de SQL Server 2025 (version 852 vers 998) et a produit 31 tables métier (hors tables d'archive).

## 4.2 Ingestion dans la couche Bronze avec DLT

### 4.2.1 Stratégie de chargement

Le pipeline `ingestion/wwi_pipeline.py` déclare une ressource DLT par table source. La stratégie est choisie automatiquement à partir des métadonnées de chaque table (tableau 4.1).

**Table 4.1** – Stratégie de chargement par type de table

| Cas | Curseur | Écriture | Tables |
|---|---|---|---|
| Table transactionnelle avec `LastEditedWhen` et clé primaire | `LastEditedWhen` | `merge` | 12 |
| Table temporelle avec `ValidFrom` et clé primaire | `ValidFrom` | `merge` | 16 |
| Table d'archive (sans clé primaire) | — | `replace` | 2 |

En mode incrémental, DLT mémorise la plus grande valeur du curseur déjà chargée et n'extrait au passage suivant que les lignes plus récentes ; ces lignes sont ensuite **fusionnées** sur la clé primaire, ce qui met à jour les lignes modifiées sans créer de doublons.

```
for full_name in tables:
    schema, name = full_name.split(".")
    columns = {c["name"] for c in inspector.get_columns(name, schema=schema)}
    pk = inspector.get_pk_constraint(name, schema=schema)["constrained_columns"]
    # Incrémental uniquement avec une clé de fusion : sans clé, un "replace"
    # incrémental écraserait la table avec les seules nouvelles lignes
    cursor = next((c for c in CURSOR_COLUMNS if c in columns), None) if pk else None
```

```
    resource = sql_table(
        credentials=engine, table=name, schema=schema,
        backend="pyarrow", reflection_level="full",
        table_adapter_callback=drop_unsupported_columns,
        incremental=dlt.sources.incremental(cursor) if cursor else None,
    ).with_name(f"{schema}_{name}")
    if cursor:
        resource.apply_hints(write_disposition="merge", primary_key=pk)
    else:
        resource.apply_hints(write_disposition="replace")
    yield resource
```

**Listing 4.1** – Choix automatique de la stratégie de chargement (extrait)

### 4.2.2 Traitements particuliers

— **Types non supportés** : les colonnes `geography` (localisations, frontières) et `varbinary` (photos, mots de passe hachés) sont retirées avant l'extraction par une fonction de rappel (`table_adapter_callback`). Elles n'ont pas d'usage analytique et certaines sont sensibles.

— **Performance** : l'extraction utilise le moteur `pyarrow`, qui transfère les données par blocs colonnes et les charge dans Snowflake au format Parquet.

— **Connexions** : DLT extrayant plusieurs tables en parallèle, le pool de connexions SQLAlchemy par défaut (5 + 10) était épuisé ; il a été remplacé par un `NullPool`.

### 4.2.3 Résultat et réconciliation

Le premier chargement complet transfère **30 tables** (près d'un million de lignes) en environ une minute. Le script `ingestion/check_bronze.py` compare, table par table, le nombre de lignes entre SQL Server et Snowflake : les 28 tables principales présentent un écart nul. Une seconde exécution n'extrait aucune ligne pour les tables incrémentales, ce qui confirme le bon fonctionnement du curseur.

**Table 4.2** – Volumes des principales tables Bronze (réconciliées avec SQL Server)

| Table | Lignes | Table | Lignes |
|---|---|---|---|
| warehouse_stock_item_transactions | 236 667 | application_cities | 37 940 |
| sales_order_lines | 231 412 | purchasing_purchase_order_lines | 8 367 |
| sales_invoice_lines | 228 265 | purchasing_supplier_transactions | 2 438 |
| sales_customer_transactions | 97 147 | purchasing_purchase_orders | 2 074 |
| sales_orders | 73 595 | application_people | 1 111 |
| sales_invoices | 70 510 | sales_customers | 663 |

## 4.3 Couche Silver : nettoyage et normalisation

La couche Silver compte **29 modèles dbt**. Les règles de nettoyage sont factorisées dans des macros réutilisables (listing 4.2) et appliquées de manière homogène à toutes les tables.

```
{# Texte nettoyé : espaces supprimés, chaînes vides -> NULL #}
{% macro clean_text(col) -%}
    nullif(trim({{ col }}), '')
```

```
{%- endmacro %}

{# Montant normalisé en décimal à 2 chiffres #}
{% macro to_amount(col) -%}
    cast({{ col }} as number(18, 2))
{%- endmacro %}

{# Dédoublonnage de sécurité : garde la version la plus récente par clé #}
{% macro dedup(key, order_col) -%}
    qualify row_number() over (partition by {{ key }} order by {{ order_col }} desc) = 1
{%- endmacro %}
```

**Listing 4.2** – Macros de nettoyage (`macros/cleaning.sql`, extrait)

**Table 4.3** – Règles appliquées dans la couche Silver

| **Règle** | **Mise en œuvre** |
|---|---|
| Texte | Suppression des espaces, chaînes vides converties en `NULL`, e-mails et URL en minuscules, codes ISO en majuscules |
| Montants | Conversion en `NUMBER(18,2)` ; taux en `NUMBER(5,3)` |
| Valeurs manquantes | Booléens à `false`, quantités à 0 |
| Dédoublonnage | Version la plus récente par clé (`QUALIFY ROW_NUMBER()`) |
| Règles métier | Montant HT = quantité × prix unitaire ; montant TTC = `extended_price` |
| Données semi-structurées | Pays de fabrication extrait du JSON `CustomFields` (`TRY_PARSE_JSON`) |
| Minimisation | Coordonnées bancaires des fournisseurs, identifiants de connexion et préférences des utilisateurs exclus |

## 4.4 Couche Gold : le schéma en étoile

La couche Gold contient **15 tables** : 6 dimensions, 6 faits et 3 tables d'historisation (tableau 4.4).

**Table 4.4** – Tables de la couche Gold

| Table | Grain et contenu | Lignes |
|---|---|---|
| `fact_sales` | Ligne de facture : CA HT, TVA, TTC, bénéfice, quantité | 228 265 |
| `fact_orders` | Ligne de commande : quantités commandées et préparées, reliquats | 231 412 |
| `fact_purchases` | Ligne de bon de commande fournisseur : quantités et montants d'achat | 8 367 |
| `fact_customer_transactions` | Transaction client : facture ou paiement, solde dû | 97 147 |
| `fact_stock_movements` | Mouvement de stock : entrée ou sortie | 236 667 |
| `fact_stock_holdings` | Produit : stock actuel, valeur, alerte de réapprovisionnement | 227 |
| `dim_date` | Jour (2013–2016), libellés français | 1 461 |
| `dim_customer` | Client : catégorie, groupement d'achat, territoire, pays | 663 |
| `dim_product` | Produit : marque, couleur, groupe, fournisseur | 227 |
| `dim_supplier` | Fournisseur : catégorie, pays, délai de paiement | 13 |
| `dim_employee` | Employé, dont commerciaux | 19 |
| `dim_city` | Ville, province, territoire commercial, pays, continent | 37 940 |
| `dim_product_history` | Version d'un produit (SCD2) : évolution des prix | 671 |
| `dim_customer_history` | Version d'un client (SCD2) : crédit, catégorie, remise | 713 |
| `fact_stock_history` | Version du niveau de stock (snapshot dbt) | 227+ |

Le listing 4.3 montre la construction de la table de faits des ventes, matérialisée de manière incrémentale (section 6.1).

```
{{ config(materialized='incremental', unique_key='invoice_line_id',
          incremental_strategy='merge', on_schema_change='append_new_columns') }}

-- Grain : une ligne de facture
select
    il.invoice_line_id, il.invoice_id, i.order_id, i.invoice_date,
    i.customer_id, il.stock_item_id, i.salesperson_person_id as salesperson_id,
    c.delivery_city_id, dm.delivery_method_name as delivery_method,
    i.is_credit_note, il.quantity, il.unit_price, il.tax_rate,
    il.amount_excluding_tax, il.tax_amount, il.amount_including_tax,
    il.line_profit as profit,
    greatest(il.last_edited_when, i.last_edited_when) as last_edited_when
from {{ ref('slv_invoice_lines') }} il
join {{ ref('slv_invoices') }} i on i.invoice_id = il.invoice_id
left join {{ ref('slv_customers') }} c on c.customer_id = i.customer_id
left join {{ ref('slv_delivery_methods') }} dm on dm.delivery_method_id = i.
    delivery_method_id
{% if is_incremental() %}
where greatest(il.last_edited_when, i.last_edited_when) >
      (select coalesce(max(last_edited_when), '1900-01-01'::timestamp_ntz) from {{ this }})
{% endif %}
```

**Listing 4.3** – Table de faits des ventes (`fact_sales.sql`, extrait)

Une macro `generate_schema_name` redéfinit le comportement par défaut de dbt afin que les modèles soient créés directement dans les schémas `SILVER` et `GOLD`, et une configuration `grants` accorde automatiquement le droit `SELECT` sur toute table Gold au rôle `WWI_ANALYST`.

## 4.5 Historisation des données (SCD de type 2)

La couche Bronze ne conserve que la dernière version de chaque ligne. Pour analyser l'évolution des données dans le temps, deux mécanismes complémentaires ont été mis en place.

### 4.5.1 Exploitation des tables temporelles de SQL Server

Les tables temporelles de WWI conservent chaque version passée dans une table d'archive. `Sales.Customers_Archive` et `Warehouse.StockItems_Archive` sont donc ingérées en Bronze, puis réunies avec la version courante dans les modèles `slv_stock_items_history` et `slv_customers_history`. Chaque version reçoit une clé de substitution (hachage MD5 de l'identifiant et de la date de début de validité). Les tables Gold `dim_product_history` (671 versions pour 227 produits) et `dim_customer_history` (713 versions pour 663 clients) exposent la période de validité, un numéro de version et un indicateur de version courante. Il devient alors possible de répondre à des questions comme « comment le prix de tel produit a-t-il évolué ? ».

### 4.5.2 Snapshot dbt du niveau de stock

La table des niveaux de stock (`StockItemHoldings`) n'est pas temporelle : la source n'en garde que la valeur courante. Un **snapshot dbt** (stratégie `check` sur la quantité, le coût, le seuil et la cible de stock) enregistre une nouvelle version à chaque changement détecté lors d'une exécution ; la table `fact_stock_history` en est issue. L'historique s'enrichit ainsi automatiquement à chaque exécution quotidienne de la plateforme.

```yaml
snapshots:
  - name: snp_stock_item_holdings
    relation: ref('slv_stock_item_holdings')
    config:
      schema: SILVER
      unique_key: stock_item_id
      strategy: check
      check_cols: [quantity_on_hand, last_cost_price, reorder_level, target_stock_level]
      hard_deletes: invalidate
```

**Listing 4.4** – Snapshot dbt du niveau de stock (`snapshots.yml`)

## 4.6 Qualité des données

dbt exécute plus de **140 tests** à chaque construction (tableau 4.5).

**Table 4.5** – Typologie des tests de qualité

| Type de test | Exemples |
|---|---|
| Unicité, non-nullité | Clé de chaque table Silver et Gold, clés de version SCD2 |
| Intégrité référentielle | Chaque fait pointe vers un client, un produit, une date, un employé existant |
| Valeurs autorisées | `direction` ∈ {Entrée, Sortie} |
| Règles métier | HT + TVA = TTC (tolérance d'un centime) ; quantités vendues strictement positives ; dates de vente couvertes par `dim_date` |
| Historisation | Exactement une version courante par produit et par client |
| Réconciliation | CA TTC de la couche Gold égal à celui de la couche Bronze |

```sql
-- Réconciliation : le CA TTC de la couche Gold égale celui de la couche Bronze
with bronze as (
    select sum(extended_price) as total from {{ source('bronze', 'sales_invoice_lines') }}
),
gold as (
    select sum(amount_including_tax) as total from {{ ref('fact_sales') }}
)
select bronze.total as bronze_total, gold.total as gold_total
from bronze, gold
where abs(bronze.total - gold.total) > 1
```

**Listing 4.5** – Test de réconciliation Bronze / Gold

Un test dbt échoue s'il renvoie au moins une ligne. La commande `dbt build` exécute les modèles et leurs tests dans l'ordre des dépendances : un modèle dont un test échoue bloque ses descendants.

## 4.7 Orchestration avec Dagster

### 4.7.1 Assets et lignage

Le fichier `orchestration/definitions.py` déclare :

— les **assets Bronze** à partir de la source DLT (`@dlt_assets`) ; un traducteur personnalisé leur attribue la clé `["bronze", <table>]`, identique à celle des *sources* dbt, ce qui relie automatiquement DLT et dbt dans le lignage ;

— les **assets Silver et Gold** à partir du manifeste dbt (`@dbt_assets`), regroupés par couche ;

— les **tests dbt**, convertis automatiquement en *asset checks*.

Au total, Dagster gère **75 assets** (30 Bronze, 30 Silver, 15 Gold) et **146 contrôles** de qualité. La figure 4.1 montre le lignage de la table `fact_sales` : on y retrouve les tables Bronze chargées par DLT, les modèles Silver et le fait Gold.

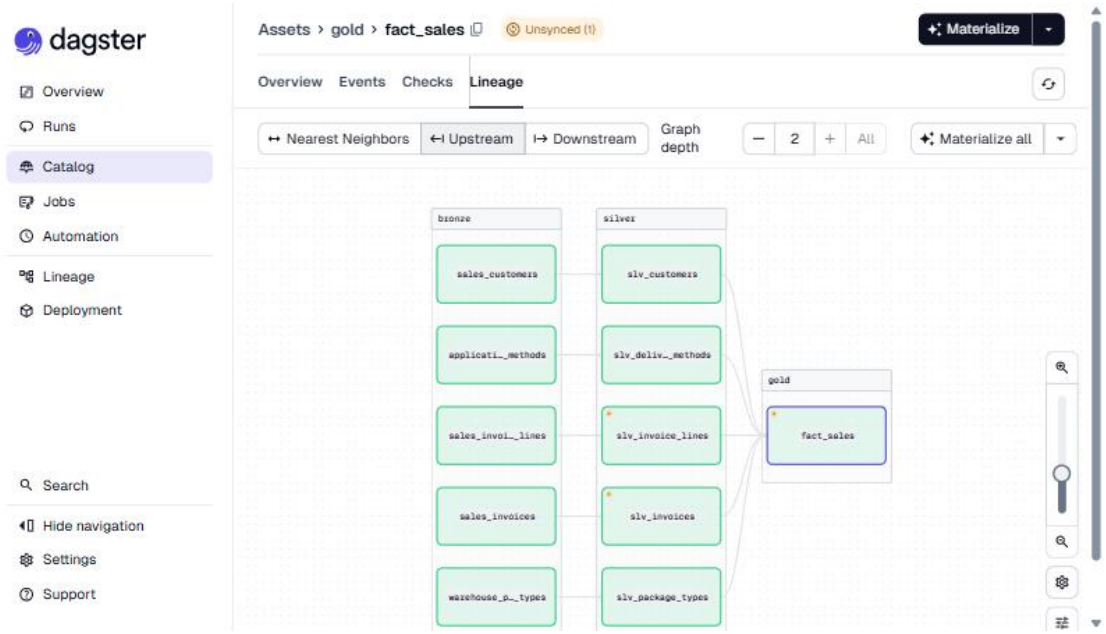



**Figure 4.1** – Lignage amont de `gold/fact_sales` dans Dagster (Bronze → Silver → Gold)

### 4.7.2 Planification et reprise sur erreur

Le job `wwi_daily_job` rejoue toute la chaîne : ingestion DLT, puis `dbt build`. Il est déclenché **chaque jour à 06 h 00 (heure de Dakar)** par le planning `wwi_daily_schedule`. Une politique de reprise (`RetryPolicy`) relance automatiquement une étape en échec jusqu'à deux fois, à 60 secondes d'intervalle, ce qui absorbe les erreurs transitoires (réseau, verrou temporaire).

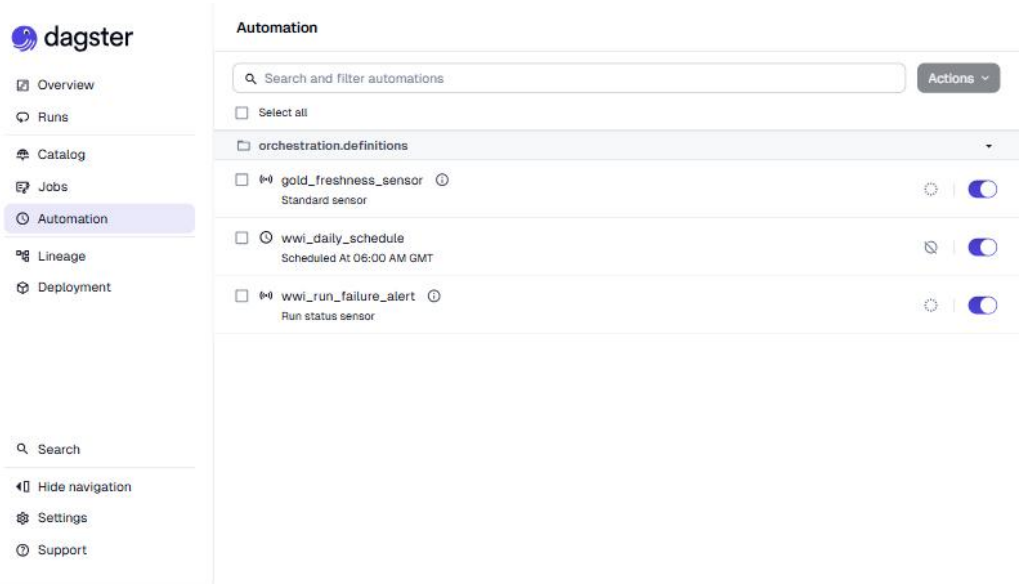



**Figure 4.2** – Automatisations actives dans Dagster : planning quotidien et capteurs d'alerte

Le premier run complet du job a réussi en environ 4 min 30 (3 min 32 pour l'étape Bronze, 53 s pour l'étape dbt).

CHAPITRE 5

# Application Text-to-SQL avec Cortex AI

## 5.1 Choix du service : Cortex Analyst

Deux approches étaient envisageables pour générer le SQL avec Snowflake Cortex :

1. appeler une fonction LLM générique (`SNOWFLAKE.CORTEX.COMPLETE`) en lui fournissant la question et une description textuelle du schéma ;
2. utiliser **Cortex Analyst**, service spécialisé guidé par un modèle sémantique.

Les tests réalisés sur le compte ont tranché la question : sur un **compte d'essai** Snowflake, toutes les fonctions LLM génériques (`COMPLETE`, `AI_COMPLETE`, `SENTIMENT`, `AI_CLASSIFY`…) sont désactivées et renvoient l'erreur « *AI function … is not available for trial accounts* ». Cortex Analyst, en revanche, est disponible. Ce choix est par ailleurs le plus pertinent techniquement : le modèle sémantique encode la connaissance métier (synonymes, calculs, jointures) et produit un SQL nettement plus fiable qu'une génération libre.

## 5.2 Le modèle sémantique

Le fichier `app/semantic_model.yaml` décrit la couche Gold pour Cortex Analyst :

— **10 tables logiques** (5 faits, 5 dimensions), chacune avec ses dimensions, dimensions temporelles, faits et métriques ;

— **des synonymes français** : « CA », « chiffre d'affaires », « ventes » pour `amount_excluding_tax` ; « marge », « bénéfice » pour `profit` ; « commercial », « vendeur » pour `employee_name`… ;

— **des valeurs d'exemple** (`sample_values`) relevées dans les données réelles, qui aident le modèle à traduire « supermarchés » en `'Supermarket'` ;

— **des métriques** : chiffre d'affaires total, bénéfice total, taux de marge, nombre de factures et de commandes ;

— **11 relations** de type plusieurs-à-un entre faits et dimensions ;

— **3 requêtes vérifiées** correspondant aux questions de l'énoncé ;

— **des instructions personnalisées** : répondre en français, considérer par défaut le CA hors taxes, limiter les classements à 10 lignes, arrondir à deux décimales.

```
- name: fact_sales
  description: Ventes facturées, une ligne par ligne de facture.
  base_table: {database: WWI_DB, schema: GOLD, table: FACT_SALES}
  primary_key: {columns: [invoice_line_id]}
  time_dimensions:
    - {name: invoice_date, expr: invoice_date, data_type: date,
```

```
      synonyms: [date de vente, date de facture, date]}
  facts:
    - {name: amount_excluding_tax, expr: amount_excluding_tax, data_type: number,
       synonyms: [chiffre d'affaires, CA, ventes, revenu, montant HT]}
    - {name: profit, expr: profit, data_type: number, synonyms: [bénéfice, marge, profit]}
  metrics:
    - {name: margin_rate, expr: "SUM(profit) / NULLIF(SUM(amount_excluding_tax), 0)",
       description: Taux de marge (bénéfice / CA HT), synonyms: [taux de marge]}
```

**Listing 5.1** – Extrait du modèle sémantique : table des ventes et métrique de marge

**Point d'attention rencontré**

Dans la syntaxe YAML compacte `{clé: valeur, ...}`, une virgule non protégée termine la valeur. L'expression `NULLIF(SUM(...), 0)` était ainsi tronquée et Cortex Analyst rejetait le modèle (« *Invalid semantic model yaml* »). Toute expression contenant une virgule doit être placée entre guillemets.

## 5.3 Fonctionnement de l'application

La figure 5.1 décrit le traitement d'une question.

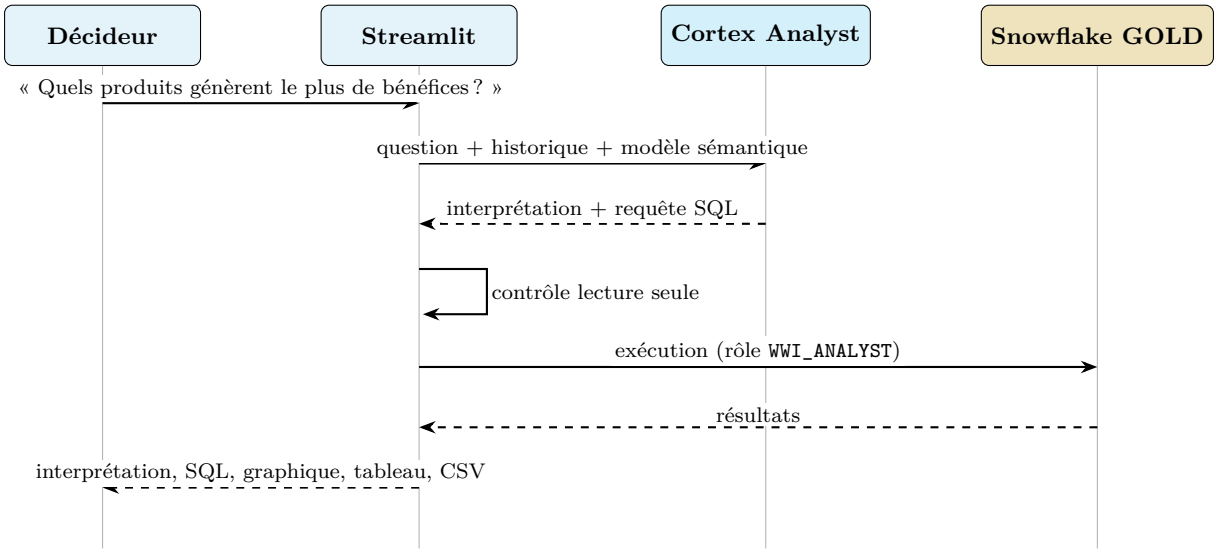



**Figure 5.1** – Traitement d'une question en langage naturel

### 5.3.1 Architecture du code

Le code est séparé en deux modules :

— `cortex_analyst.py` : le *moteur*, indépendant de l'interface. Il envoie la conversation à Cortex Analyst, contrôle et exécute le SQL. Il propose deux implémentations interchangeables : `LocalBackend` (connecteur Python et identifiants du fichier `.env`) et `SnowflakeBackend` (session active de Streamlit in Snowflake, sans identifiant à gérer) ;

— `streamlit_app.py` : l'*interface* de conversation.

### 5.3.2 Conversation multi-tours

L'historique complet des échanges est transmis à Cortex Analyst à chaque question. Les questions de suivi sont ainsi interprétées dans leur contexte : après « Quel est le chiffre d'affaires par catégorie de client et par année ? », la question « Et seulement pour les supermarchés, par mois en 2015 ? » est correctement comprise comme « *le chiffre d'affaires HT par mois en 2015, uniquement pour la catégorie de client Supermarket* ».

### 5.3.3 Choix automatique de la visualisation

L'application analyse le résultat pour choisir la représentation la plus adaptée :

— une valeur unique est affichée comme un **indicateur** (KPI) ;

— si une colonne est temporelle (date, mois, année…), une **courbe** est tracée, avec une série par catégorie si une seconde dimension est présente ;

— sinon, un **diagramme en barres** horizontal trié par valeur décroissante est affiché.

Les colonnes numériques représentant le temps (année, mois) ou des identifiants sont traitées comme des axes et non comme des mesures.

## 5.4 Sécurité de la génération de SQL

Confier l'écriture du SQL à un modèle de langage impose des garde-fous. Trois niveaux de protection se cumulent :

1. **Contrôle syntaxique** avant exécution (listing 5.2) : une seule instruction, commençant par `SELECT` ou `WITH`, sans aucun mot-clé d'écriture ou d'administration ;
2. **Moindre privilège** : la requête est exécutée avec le rôle `WWI_ANALYST`, qui ne peut que lire le schéma Gold ;
3. **Limitation des volumes** : 10 000 lignes au maximum sont rapatriées.

```python
FORBIDDEN = re.compile(
    r"\b(insert|update|delete|merge|drop|create|alter|truncate|grant|revoke"
    r"|call|copy|put|remove|use|execute)\b", re.IGNORECASE)

def check_read_only(sql: str) -> None:
    """Refuse toute requête qui n'est pas une simple lecture."""
    stripped = re.sub(r"--[^\n]*|/\*.*?\*/", "", sql, flags=re.DOTALL).strip().rstrip(";")
    if ";" in stripped:
        raise ValueError("Une seule instruction SQL est autorisée.")
    if not re.match(r"^(with|select)\b", stripped, re.IGNORECASE):
        raise ValueError("Seules les requêtes SELECT sont autorisées.")
    if FORBIDDEN.search(stripped):
        raise ValueError("La requête contient une instruction non autorisée.")
```

**Listing 5.2** – Contrôle de lecture seule du SQL généré

## 5.5 Interface utilisateur

L'interface reprend l'identité visuelle de l'Université Cheikh Anta Diop : un en-tête présente le logo, le nom de l'université et de la faculté, et un pied de page rappelle le département, la

section et la formation. La barre latérale propose des questions d'exemple et la réinitialisation de la conversation.

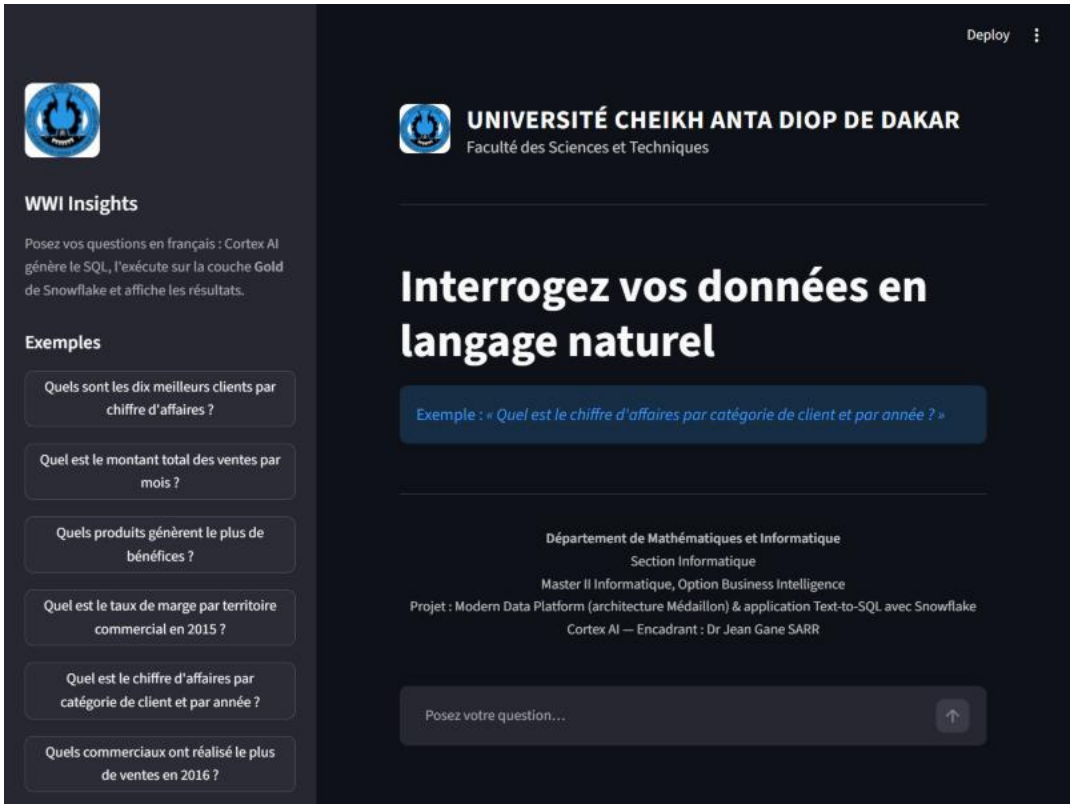



**Figure 5.2** – Page d'accueil de l'application : en-tête UCAD, exemples de questions et pied de page

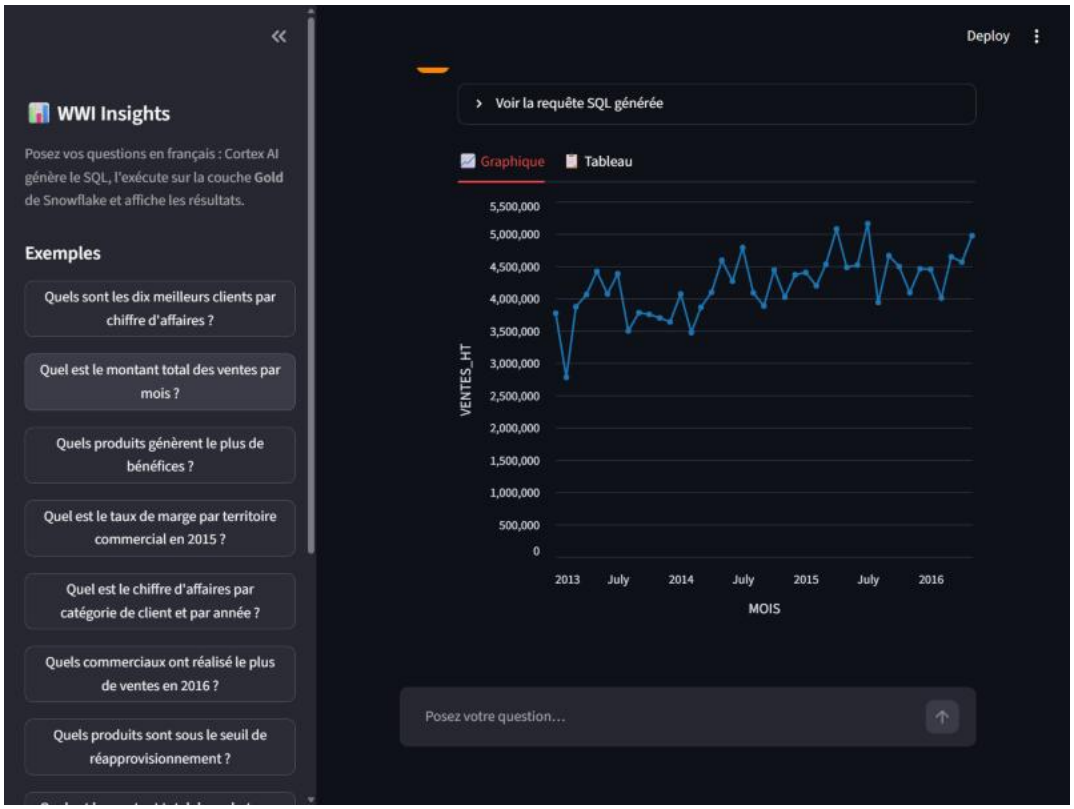



**Figure 5.3** – « Quel est le montant total des ventes par mois ? » : courbe générée automatiquement

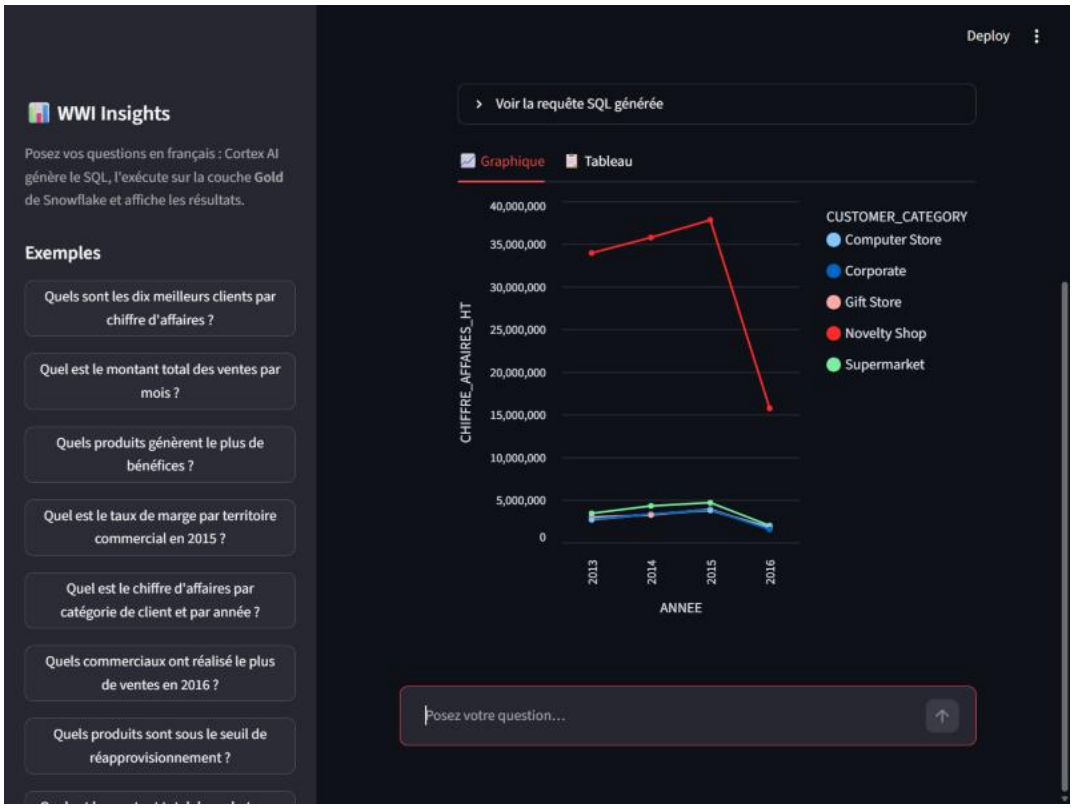



**Figure 5.4** – « Quel est le chiffre d'affaires par catégorie de client et par année ? » : une série par catégorie

Pour chaque réponse, l'utilisateur dispose de l'interprétation de sa question par Cortex Analyst, de la requête SQL générée (dans un panneau dépliable, pour la transparence et l'apprentissage), du graphique, du tableau de résultats et d'un export CSV. En cas d'ambiguïté, Cortex Analyst propose des reformulations cliquables.

CHAPITRE 6

# Industrialisation de la plateforme

Une fois la chaîne fonctionnelle, plusieurs limites ont été identifiées : absence d'historique, reconstruction complète de toutes les tables à chaque exécution, application dépendante d'un poste local, absence d'alerte en cas d'incident et validation manuelle des modifications. Ce chapitre présente les réponses apportées. L'historisation ayant été décrite à la section 4.5, il traite les quatre autres points.

## 6.1 Traitements incrémentaux

Dans la version initiale, `dbt build` reconstruisait intégralement toutes les tables. Ce coût, négligeable sur WWI, devient prohibitif sur des volumes réels. Les **dix modèles les plus volumineux** sont désormais matérialisés en mode `incremental` avec la stratégie `merge` (tableau 6.1).

**Table 6.1** – Modèles incrémentaux

| Couche | Modèle | Clé de fusion |
|---|---|---|
| Silver | `slv_invoice_lines` | `invoice_line_id` |
| | `slv_invoices` | `invoice_id` |
| | `slv_order_lines` | `order_line_id` |
| | `slv_orders` | `order_id` |
| | `slv_stock_item_transactions` | `stock_item_transaction_id` |
| | `slv_customer_transactions` | `customer_transaction_id` |
| Gold | `fact_sales` | `invoice_line_id` |
| | `fact_orders` | `order_line_id` |
| | `fact_stock_movements` | `stock_item_transaction_id` |
| | `fact_customer_transactions` | `customer_transaction_id` |

Une macro `incremental_filter` ne retient que les lignes dont la date de dernière modification est postérieure à la plus récente déjà présente dans la table cible. Pour les faits issus d'une jointure en-tête/ligne, la date retenue est la plus récente des deux (`GREATEST`), de sorte qu'une modification de l'en-tête (par exemple la date de livraison d'une facture) est aussi propagée.

**Vérification**

Après une reconstruction complète (`dbt build --full-refresh`), une exécution normale des dix modèles incrémentaux a traité **0 ligne** au lieu de 228 265 pour `fact_sales` : seules les modifications seront désormais retraitées.

## 6.2 Alertes et supervision

### 6.2.1 Alertes Dagster

Le module `orchestration/alerts.py` ajoute deux capteurs actifs par défaut :

— `wwi_run_failure_alert` : un *run failure sensor* déclenché dès qu'un run échoue. Il construit un message (job, identifiant du run, étapes en échec, erreur, lien vers l'interface) et l'envoie sur **Slack** (webhook) et/ou par **e-mail** (SMTP) selon la configuration du fichier `.env` ; à défaut, l'alerte est journalisée ;

— `gold_freshness_sensor` : des **contrôles de fraîcheur** vérifient que les tables Gold principales ont été rafraîchies au cours des 26 dernières heures (exécution quotidienne plus une marge de deux heures). Un planning bloqué ou un démon arrêté est ainsi détecté.

### 6.2.2 Maîtrise des coûts Snowflake

Un **moniteur de ressources** (`WWI_CREDIT_MONITOR`) est associé à l'entrepôt `WWI_WH` : il envoie une notification à 75 % et 90 % du quota mensuel de crédits, suspend l'entrepôt à 100 % et l'arrête immédiatement à 110 %. Il protège en particulier les crédits limités du compte d'essai.

## 6.3 Déploiement dans Streamlit in Snowflake

L'application, initialement exécutée sur le poste du développeur, a été rendue déployable dans **Streamlit in Snowflake**. Ce mode d'hébergement présente plusieurs avantages :

— aucun serveur ni identifiant à gérer : l'application utilise la session de l'utilisateur connecté à Snowsight ;

— les données ne quittent pas Snowflake ;

— le partage se fait par les mécanismes de droits de Snowflake.

Le script `deploy/deploy_sis.py` automatise le déploiement :

1. création du schéma `WWI_DB.APP` et attribution au rôle `WWI_ANALYST` des droits de création de stage et d'application ;
2. création d'un stage interne et envoi des fichiers de l'application (`PUT`) : code, modèle sémantique, logo et fichier `environment.yml` des dépendances ;
3. création de l'objet `STREAMLIT WWI_DB.APP.WWI_INSIGHTS` avec l'entrepôt `WWI_WH`.

L'application appartient au rôle `WWI_ANALYST` : elle s'exécute donc avec ses seuls droits de lecture sur Gold. Le même code source fonctionne dans les deux environnements, le moteur détectant la présence du module interne `_snowflake` pour choisir entre l'appel REST local et l'API interne `_snowflake.send_snow_api_request`.

## 6.4 Intégration et déploiement continus

Le dépôt est hébergé sur GitHub. Le workflow `.github/workflows/ci.yml` s'exécute à chaque *push* et à chaque *pull request* (figure 6.1).

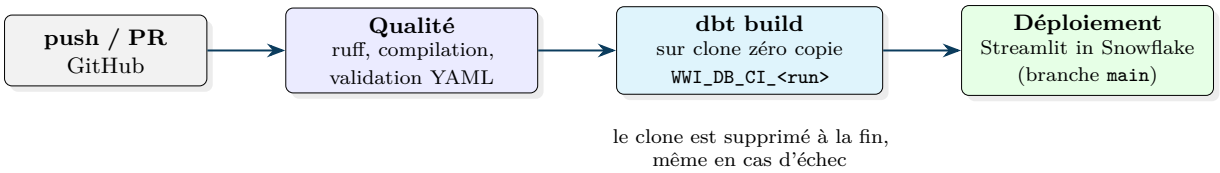



**Figure 6.1** – Chaîne d'intégration et de déploiement continus

1. **Qualité** : analyse statique du code Python (`ruff`), compilation de tous les scripts, validation syntaxique de tous les fichiers YAML et de la structure du modèle sémantique.
2. **dbt** : la base de production est clonée par **clonage zéro copie** (`CREATE DATABASE WWI_DB_CI_<run> CLONE WWI_DB`) : le clone est instantané et ne duplique aucune donnée. `dbt build` y est exécuté (modèles, snapshot et tests), puis le clone est supprimé. Les modifications sont ainsi testées sur des données réelles sans jamais toucher à la production. La base cible étant paramétrée par la variable `SNOWFLAKE_DATABASE`, aucun code n'est spécifique à la CI.
3. **Déploiement** : sur la branche `main`, si les étapes précédentes réussissent, l'application Streamlit in Snowflake est redéployée.

Les identifiants Snowflake sont stockés dans les *secrets* chiffrés du dépôt GitHub, jamais dans le code.

CHAPITRE 7

# Résultats et évaluation

## 7.1 Bilan quantitatif de la plateforme

**Table 7.1** – Indicateurs de la plateforme réalisée

| Élément | Valeur |
|---|---|
| Tables ingérées en Bronze (dont 2 tables d'archive) | 30 |
| Lignes chargées (premier chargement) | ≈ 1 million |
| Modèles Silver / snapshot | 29 / 1 |
| Tables Gold (dimensions, faits, historisation) | 15 |
| Nœuds dbt exécutés par `dbt build` (modèles, snapshot, tests) | 189 |
| Tests dbt (tous au vert) | 144 |
| Assets Dagster / contrôles de qualité | 75 / 146 |
| Écart de volume SQL Server / Bronze | 0 ligne |
| Écart de CA TTC Bronze / Gold | 0 |
| Durée d'une exécution complète (ingestion + dbt) | ≈ 4 min 30 |
| Lignes retraitées par une exécution incrémentale sans changement | 0 |

## 7.2 Évaluation de l'application Text-to-SQL

Un script de test de bout en bout (`app/test_questions.py`) soumet une série de questions à Cortex Analyst, exécute le SQL obtenu et affiche les résultats. Les réponses ont été comparées à des requêtes SQL écrites manuellement : **toutes les questions testées ont produit un SQL exécutable et des résultats identiques** à la référence (tableau 7.2).

**Table 7.2** – Questions testées et résultats obtenus

| Question posée en français | Résultat (extrait) |
|---|---|
| Quels sont les dix meilleurs clients par chiffre d'affaires ? | 1er : Tailspin Toys (Inguadona, MN) avec 381 585,35 $ HT ; puis Tailspin Toys (Minidoka, ID) avec 371 822,30 $ et Mauno Laurila avec 369 058,30 $. |
| Quel est le montant total des ventes par mois ? | 41 mois, de janvier 2013 (3 770 410,85 $) à mai 2016 ; courbe affichée automatiquement. |
| Quels produits génèrent le plus de bénéfices ? | « 20 mm Double sided bubble wrap 50m » : 5 293 680 $ ; « Air cushion machine (Blue) » : 4 439 391 $. |

| Question posée en français | Résultat (extrait) |
|---|---|
| Quel est le taux de marge par territoire commercial en 2015 ? | 9 territoires ; de 50,57 % (Great Lakes) à environ 49,9 % ; utilisation de la métrique `margin_rate`. |
| Quels produits sont sous le seuil de réapprovisionnement ? | 2 produits, dont « Shipping carton (Brown) 305x305x305mm » (38 unités pour un seuil de 50). |
| Quel est le chiffre d'affaires par pays ? | United States : 172 261 341,20 $ HT (tous les clients de WWI sont américains). |
| Quel est le chiffre d'affaires par catégorie de client et par année ? | Novelty Shop domine chaque année (environ 38 M$ en 2015) ; une série par catégorie. |
| *(suivi)* Et seulement pour les supermarchés, par mois en 2015 ? | Question interprétée dans le contexte : CA HT mensuel 2015 de la catégorie `Supermarket`. |

## 7.3 Quelques enseignements métier

Les données de la couche Gold permettent de dégager rapidement une vue d'ensemble de l'activité de Wide World Importers :

- un chiffre d'affaires cumulé de **172,3 M$ HT** sur 70 510 factures, pour un bénéfice de **85,7 M$**, soit un taux de marge d'environ **49,8 %** ;
- une croissance régulière du CA : 45,7 M$ en 2013, 49,9 M$ en 2014 et 54,0 M$ en 2015 (l'année 2016 ne couvre que cinq mois) ;
- un taux de marge très homogène entre territoires commerciaux (entre 49,9 % et 50,6 % en 2015) ;
- une forte concentration des achats sur deux fournisseurs (Fabrikam, Inc. et Litware, Inc.).

**Remarque sur les données**

Dans WWI, le montant total des achats fournisseurs (environ 946 M$, cohérent avec les factures fournisseurs de la source : 944 M$) dépasse largement le chiffre d'affaires. Il s'agit d'une caractéristique connue de ce jeu de données synthétique, et non d'une erreur de transformation. De même, la plupart des versions archivées des produits datent de la fin de la période (31 mai 2016) et correspondent à des mises à jour techniques : le mécanisme d'historisation est en place et sera pleinement exploité sur des données réelles.

## 7.4 Difficultés rencontrées et solutions

**Table 7.3** – Principales difficultés et solutions apportées

| Difficulté | Solution |
|---|---|
| Accès refusé du service SQL Server au fichier de sauvegarde | Copie du `.bak` dans un dossier partagé lisible par le compte de service. |

| Difficulté | Solution |
|---|---|
| Certificat non approuvé avec le pilote ODBC 18 | Option `TrustServerCertificate` (`-C`) pour l'instance locale. |
| Épuisement du pool de connexions SQLAlchemy lors de l'extraction parallèle | Utilisation d'un `NullPool` (une connexion par table extraite). |
| Types `geography` et `varbinary` non transférables | Exclusion des colonnes avant extraction (`table_adapter_callback`). |
| Risque de perte de données : table sans clé chargée en mode incrémental `replace` | Incrémental activé uniquement pour les tables dotées d'une clé primaire ; tables d'archive en rechargement complet. |
| Fonctions LLM de Cortex indisponibles sur un compte d'essai | Recours à Cortex Analyst et à un modèle sémantique, solution par ailleurs plus fiable. |
| Modèle sémantique rejeté (expression tronquée par une virgule en YAML) | Mise entre guillemets des expressions contenant une virgule ; validation YAML ajoutée en CI. |
| Graphique erroné : l'année, numérique, prise pour une mesure | Les colonnes temporelles et les identifiants sont traités comme des axes. |
| `dbt` introuvable par Dagster lorsque l'environnement virtuel n'est pas activé | Ajout du dossier de l'environnement virtuel au `PATH` au chargement des définitions. |

## 7.5 Limites

— Le compte Snowflake est un compte d'essai (30 jours, 400 $ de crédits) : les fonctions LLM génériques n'y sont pas disponibles.

— Le système source est une instance SQL Server locale : en production, l'ingestion serait exécutée sur un serveur ou un conteneur ayant accès à la source et à Snowflake.

— La qualité de la génération SQL dépend du modèle sémantique ; les questions portant sur des notions non décrites (par exemple les températures des entrepôts frigorifiques, non ingérées) ne peuvent pas recevoir de réponse.

— L'exécution de la CI sur la base clonée suppose l'octroi préalable du privilège `CREATE DATABASE` au rôle `WWI_ENGINEER` (script `snowflake/01_ameliorations.sql`).

# Conclusion générale et perspectives

Ce projet avait pour objectif de concevoir une Modern Data Platform capable d'automatiser l'ingestion, la transformation et la modélisation des données, et d'offrir aux décideurs une interface leur permettant d'interroger ces données en langage naturel.

Cet objectif est atteint. Les données transactionnelles de Wide World Importers sont extraites de SQL Server par **DLT** de manière incrémentale et chargées dans **Snowflake**, où elles sont organisées selon une **architecture Médaillon**. La couche Bronze conserve une copie fidèle et réconciliée de la source ; la couche Silver applique des règles de nettoyage homogènes et factorisées ; la couche Gold expose un schéma en étoile complété de dimensions historisées en SCD de type 2. Les transformations, développées avec **dbt**, sont versionnées, documentées et couvertes par plus de 140 tests. L'ensemble est orchestré par **Dagster**, qui offre un lignage de bout en bout, une exécution quotidienne avec reprise automatique, des contrôles de fraîcheur et des alertes.

L'application **Streamlit**, adossée à **Snowflake Cortex Analyst** et à un modèle sémantique riche, répond correctement aux questions de l'énoncé et à des questions plus élaborées, y compris des questions de suivi, en restituant le SQL généré, un graphique adapté et le tableau des résultats. Sa conception place la sécurité au premier plan : contrôle de lecture seule, rôle aux droits minimaux et absence d'identifiants dans le code. Déployée aussi dans **Streamlit in Snowflake** et validée par une chaîne d'**intégration continue** sur clone zéro copie, la plateforme dépasse le stade du prototype pour s'approcher d'une solution industrialisée.

Sur le plan personnel, ce projet m'a permis de mettre en pratique l'ensemble de la chaîne de valeur de la donnée — ingénierie, modélisation dimensionnelle, qualité, orchestration, sécurité, DevOps — et de mesurer l'apport, mais aussi les exigences, de l'intelligence artificielle générative appliquée à l'analyse de données : un LLM n'est fiable que s'il s'appuie sur un modèle de données propre et sur une sémantique métier explicitement décrite.

## Perspectives

Plusieurs prolongements sont envisageables :

- **Enrichissement du modèle sémantique** : nouvelles requêtes vérifiées issues des questions réelles des utilisateurs, publication sous forme de *semantic view* Snowflake, et couverture des historisations SCD2 et des températures d'entrepôt ;
- **Retour utilisateur** : collecte des appréciations (réponse correcte ou non) pour mesurer en continu la précision du Text-to-SQL ;
- **Ingestion en quasi temps réel** : exploitation de la capture des changements (CDC) de SQL Server ou d'un ordonnancement plus fréquent ;
- **Gouvernance** : masquage dynamique des données personnelles, politiques d'accès par ligne (*row access policies*) selon le territoire commercial de l'utilisateur, catalogue de données ;
- **Observabilité** : tableau de bord de suivi des coûts Snowflake, des durées d'exécution et des résultats des tests ;
- **Restitution** : tableaux de bord prédéfinis (Power BI, Streamlit) pour les indicateurs récurrents, l'interface conversationnelle restant dédiée aux questions ad hoc.

ANNEXE A

# Structure du dépôt

Le code source est disponible sur GitHub :

`https://github.com/mouhameddiop06/wwi-modern-data-platform`.

```
projet_pratique_bi/
|-- .env.example              Modèle des paramètres de connexion
|-- requirements.txt          Dépendances Python
|-- pyproject.toml            Configuration du lint (ruff)
|-- dbt.ps1 / dagster.ps1 / streamlit.ps1   Scripts de lancement
|-- snowflake/
|   |-- 00_setup.sql          Entrepôt, base, schémas, rôles, utilisateur
|   `-- 01_ameliorations.sql  Moniteur de crédits, droits CI, schéma APP
|-- ingestion/
|   |-- wwi_pipeline.py       Pipeline DLT SQL Server -> BRONZE
|   `-- check_bronze.py       Réconciliation des volumes
|-- transform/wwi_dbt/
|   |-- dbt_project.yml, profiles.yml
|   |-- macros/               clean_text, to_amount, dedup, surrogate_key,
|   |                         incremental_filter, generate_schema_name
|   |-- models/silver/        29 modèles slv_* + sources
|   |-- models/gold/          dimensions, faits, historisation + tests
|   |-- snapshots/            snapshot du niveau de stock
|   `-- tests/                tests métier personnalisés
|-- orchestration/
|   |-- definitions.py        Assets DLT + dbt, job, planning
|   `-- alerts.py             Alertes d'échec et contrôles de fraîcheur
|-- app/
|   |-- streamlit_app.py      Interface utilisateur
|   |-- cortex_analyst.py     Moteur Text-to-SQL (local / Snowflake)
|   |-- semantic_model.yaml   Modèle sémantique de la couche GOLD
|   |-- environment.yml       Dépendances Streamlit in Snowflake
|   `-- test_questions.py     Test de bout en bout
|-- deploy/deploy_sis.py      Déploiement Streamlit in Snowflake
|-- ci/snowflake_clone.py     Clone zéro copie pour la CI
|-- .github/workflows/ci.yml  Intégration et déploiement continus
`-- rapport/                  Ce rapport (LaTeX)
```

**Listing A.1** – Arborescence du projet

ANNEXE B

# Guide d'installation et commandes

## Installation

1. Restaurer `WideWorldImporters-Full.bak` dans SQL Server (2016 SP1 ou ultérieur).
2. Créer un compte Snowflake (édition Enterprise) et exécuter `snowflake/00_setup.sql`, puis `snowflake/01_ameliorations.sql`, en `ACCOUNTADMIN`.
3. Copier `.env.example` en `.env` et renseigner l'identifiant du compte et le mot de passe de l'utilisateur technique.
4. Créer l'environnement Python : `python -m venv .venv`, puis `.venv\Scripts\python.exe -m pip install -r requirements.txt`.

## Commandes principales

| **Objectif** | **Commande** |
|---|---|
| Ingestion incrémentale Bronze | `python ingestion/wwi_pipeline.py` |
| Rechargement complet | `python ingestion/wwi_pipeline.py --full-refresh` |
| Réconciliation des volumes | `python ingestion/check_bronze.py` |
| Construire et tester Silver et Gold | `.\dbt.ps1 build` |
| Documentation et lignage dbt | `.\dbt.ps1 docs generate` |
| Interface Dagster et planning | `.\dagster.ps1` (port 3000) |
| Application en local | `.\streamlit.ps1` (port 8501) |
| Test des questions de référence | `python app/test_questions.py` |
| Déploiement Streamlit in Snowflake | `python deploy/deploy_sis.py` |

ANNEXE C

# Extraits de code complémentaires

```python
class BronzeTranslator(DagsterDltTranslator):
    def get_asset_spec(self, data: DltResourceTranslatorData) -> AssetSpec:
        spec = super().get_asset_spec(data)
        return spec.replace_attributes(
            key=AssetKey(["bronze", snake(data.resource.name)]),  # = clé des sources dbt
            deps=[],
            group_name="bronze",
            kinds={"dlt", "sqlserver", "snowflake"},
        )

@dlt_assets(dlt_source=wwi_source(), dlt_pipeline=make_pipeline(progress=None),
            name="wwi_bronze", dagster_dlt_translator=BronzeTranslator())
def bronze_assets(context: AssetExecutionContext, dlt: DagsterDltResource):
    yield from dlt.run(context=context)
```

**Listing C.1** – Assets Bronze : relier DLT et dbt dans le lignage Dagster

```python
wwi_daily_job = define_asset_job(
    name="wwi_daily_job",
    selection=AssetSelection.all(),
    op_retry_policy=RetryPolicy(max_retries=2, delay=60),  # reprise sur erreur
)
wwi_daily_schedule = ScheduleDefinition(
    name="wwi_daily_schedule", job=wwi_daily_job,
    cron_schedule="0 6 * * *", execution_timezone="Africa/Dakar",
    default_status=DefaultScheduleStatus.RUNNING,
)
```

**Listing C.2** – Job quotidien avec reprise sur erreur

```sql
select
    h.stock_item_version_id, h.stock_item_id,
    h.stock_item_name as product_name, h.unit_price, h.recommended_retail_price,
    h.valid_from,
    case when h.valid_to >= '9999-01-01' then null else h.valid_to end as valid_to,
    h.valid_to >= '9999-01-01' as is_current,
    row_number() over (partition by h.stock_item_id order by h.valid_from) as version_number
from {{ ref('slv_stock_items_history') }} h
```

**Listing C.3** – Dimension produit historisée (SCD2)

```python
def ask(self, messages: list[dict]) -> AnalystAnswer:
    resp = _snowflake.send_snow_api_request(
        "POST", "/api/v2/cortex/analyst/message", {}, {},
        {"messages": messages, "semantic_model": SEMANTIC_MODEL},
        None, 120_000,
    )
    body = json.loads(resp["content"])
    if resp["status"] >= 400:
        raise RuntimeError(f"Cortex Analyst a répondu {resp['status']}")
    return parse_answer(body)
```

**Listing C.4** – Appel de Cortex Analyst dans Streamlit in Snowflake

# Bibliographie


[1] R. Kimball, M. Ross, *The Data Warehouse Toolkit : The Definitive Guide to Dimensional Modeling*, 3e édition, Wiley, 2013.

[2] W. H. Inmon, *Building the Data Warehouse*, 4e édition, Wiley, 2005.

[3] J. Reis, M. Housley, *Fundamentals of Data Engineering*, O'Reilly Media, 2022.

[4] Databricks, *What is a medallion architecture ?* `https://www.databricks.com/glossary/medallion-architecture`

[5] Microsoft, *Wide World Importers sample databases for Microsoft SQL.* `https://learn.microsoft.com/sql/samples/wide-world-importers-what-is`

[6] Snowflake, *Snowflake Documentation.* `https://docs.snowflake.com`

[7] Snowflake, *Cortex Analyst* et *Cortex Analyst semantic model specification.* `https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-analyst`

[8] Snowflake, *About Streamlit in Snowflake.* `https://docs.snowflake.com/en/developer-guide/streamlit/about-streamlit`

[9] dltHub, *dlt documentation — SQL database source, incremental loading.* `https://dlthub.com/docs`

[10] dbt Labs, *dbt Developer Hub — models, tests, snapshots, incremental models.* `https://docs.getdbt.com`

[11] Dagster Labs, *Dagster documentation — software-defined assets, dagster-dbt, dagster-dlt.* `https://docs.dagster.io`

[12] Streamlit, *Streamlit documentation.* `https://docs.streamlit.io`

[13] J. Li et al., *Can LLM Already Serve as A Database Interface ? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD)*, NeurIPS, 2023.